



FADO — Next Steps

Background on the tools and data, and a plan for the work ahead

Tiago Silva

CISUC · Data-Centric AI Lab

Department of Informatics Engineering, University of Coimbra



Outline

1. Current state
2. Background: the tools and data
3. Plan
4. Constraints

1. Current state

Current state

Done

Intersectional extension (composite group, per-node fairness)
Folktables sweep complete and significance-tested
Controller: warn / confirm / recover, dual-ADWIN
Statistics independently verified (scipy + statsmodels)

Missing

Only two datasets
COMPAS drift is hand-made (not standard)
Controller not yet ablated
Fairness weight λ is a single global constant

The next steps target these gaps: more standard drift, more data, and a better fairness weight.

2. Background: the tools and data

CapyMOA

A Python library for machine learning on data streams, built on MOA (Java) with a PyTorch bridge. It provides streaming classifiers (e.g. ARF), drift detectors (ADWIN), prequential evaluation, dataset loaders, and drift generators.

Does it improve performance?

Base learner: no. It does not contain our forest, so it cannot make the model itself more accurate.

Detector: yes — it is swappable. The controller reacts to a drift signal, so we can replace ADWIN with a better detector (SEED, STEPD, DDM, HDDM, ...) and benchmark it.

Baselines: yes. Standard, tested, efficient ARF and streaming trees.

Can it make deeper trees affordable?

The real cost: FADO's forest is our own implementation, so more depth means a lot more training time — that is what makes sweeping tree height expensive today.

CapyMOA does not run our forest, so it cannot speed it up — its streaming learners are different algorithms.

To afford deeper trees the lever is our own code: vectorise across trees / nodes (depth is parallel width) and parallelise the sweep. GPUs help only if we batch updates over a window — one-at-a-time online steps are too small to fill one.

Speeding up our own training

The forest is small — almost all the per-step cost is overhead, not math. These changes cut it, and **none change our results**.

Main upgrade — analytic fairness gradient.

Every step now runs a *second* backward pass (a persistent tape) just to read $\partial n / \partial W$ and $\partial n / \partial b$ for the fairness penalty. But a node is $n = \sigma((w^\top x + b) / \tau)$, so these are closed-form:

$$\frac{\partial n}{\partial W} = x \sigma'(z)^\top / \tau, \quad \frac{\partial n}{\partial b} = \sigma'(z) / \tau, \quad z = (w^\top x + b) / \tau.$$

Replacing the extra autodiff pass with this formula drops about **half the backward work** per step — identical numbers, confirmed by a gradient check.

- **Pure-NumPy fairness math** — the state is already NumPy; drop the per-step `tf.convert_to_tensor`.
- **Incremental fairness window** — $O(1)$ running counters instead of recomputing the whole window each step.
- **Graph the forward** — wrap forward + loss in `@tf.function` (eager today) · batch the trees instead of a Python loop.

All speed-only — this is what makes sweeping tree depth and the 1M-row BAF runs affordable.

Drift generation (KDD'26)

A method (Cerqueira et al., KDD'26 — same group as CopyMOA) to inject controlled, labelled concept drift into a real data stream: shuffle the stream, pick a random onset point, apply a typed change, either abrupt or gradual.

- **Class-prior** — drop a class with some probability
- **Class-label swap** — relabel one class to another
- **Feature permutation** — reorder the features
- **Feature filtering** — drop rows above a threshold

Our COMPAS drift is already a class-label swap — adopting this makes it standard and citable.

Note: it is drift/accuracy focused; its own datasets have no protected attributes.

New datasets

Feedzai BAF

Bank-account-fraud suite (NeurIPS'22).

About 1M rows per variant, 8 months of temporal shift, a protected attribute (age), and $\sim 1\%$ positives (imbalanced).

A real fairness + drift benchmark. Large, so full runs are compute-heavy.

USP / Insects

Real streams with known, labelled drift (abrupt / gradual / incremental / recurring).

No protected attributes, so it validates drift adaptation only, not fairness.

Small and cheap to run.

Per-node / per-tree λ

Today the fairness weight λ in $\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \mathcal{L}_{\text{fair}}$ is a single global constant.

Idea: let λ vary **per node or per tree** — larger where a node's group disparity is larger, smaller where fairness is already met. Each node already tracks group statistics, so it fits naturally.

Goal: a better accuracy / fairness trade-off. The benefit is uncertain and needs an ablation to confirm it beats a global λ .

3. Plan

Combining drift generation with fairness data

The drift framework is accuracy-focused; BAF and Folktables carry fairness. The two can be combined.

Inject a typed, labelled drift (from the KDD'26 method) onto a **protected subgroup** of a fairness dataset — for example, a class-label swap that affects one group only.

This gives controlled fairness-under-drift scenarios with **known onset points**: we can measure when fairness breaks and how the controller responds.

This combines the drift generator, a real dataset, and the controller into one experiment that existing benchmarks do not provide.

Order of work

	Step	Work
A	Consolidate	Finish COMPAS intersectional · controller ablation · run USP/Insects
B	Infrastructure	Adopt CappyMOA: baselines, streams, and swap in better drift detectors (SEED / STEPD) · add KDD'26 drift generation
C	New data	Build the Feedzai BAF loader · run sweeps on a suitable machine
D	Method	Per-node / per-tree adaptive λ , with an ablation

Steps A and B are mostly coding and need little compute. Steps C and D need heavier runs, so they are built first and run on a suitable machine.

4. Constraints

Constraints

Compute. The heavy multi-seed sweeps (BAF, drift Monte-Carlo, λ tuning) need a machine that can sustain the load; the home setup cannot. Coding and single-seed tests can be done anywhere.

CapyMOA is pre-1.0. Its API may still change, so we wrap only the parts we need.

BAF demographics. Its usable protected attribute is mostly age — good for marginal and temporal drift, limited for intersectional subgroups.

Adaptive λ . Its benefit is uncertain — kept last and behind an ablation.

Summary

Consolidate and make the drift standard first (low compute); then add a real fairness + drift dataset, and try the adaptive λ .

Discussion — which step to start with?

Tiago Silva · CISUC Data-Centric AI Lab · FADO